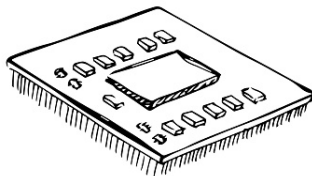


A flexible and polynomial framework for integer arithmetic in CKKS

Lorenzo Roveda

Politecnico di Torino



CrypTOgraphy days — May 22nd, 2026

Overview

1. Defining the problem
2. Our contribution
 - 2.1 Flexbile-CKKS
 - 2.2 Arithmetic primitives
3. Some experiments
4. References

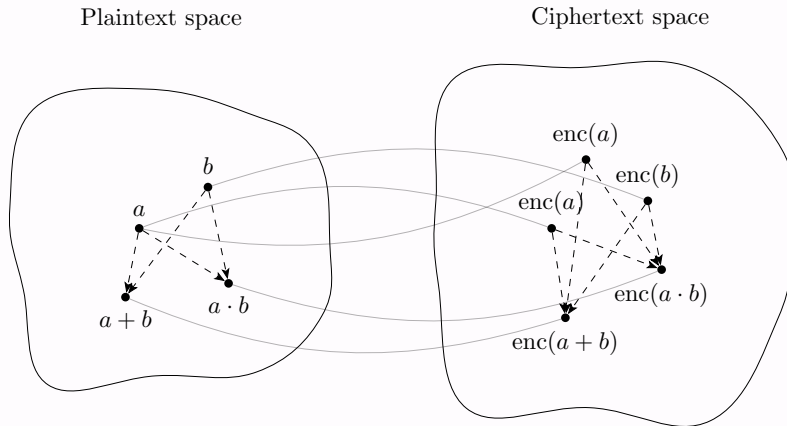
Defining the problem

Why?

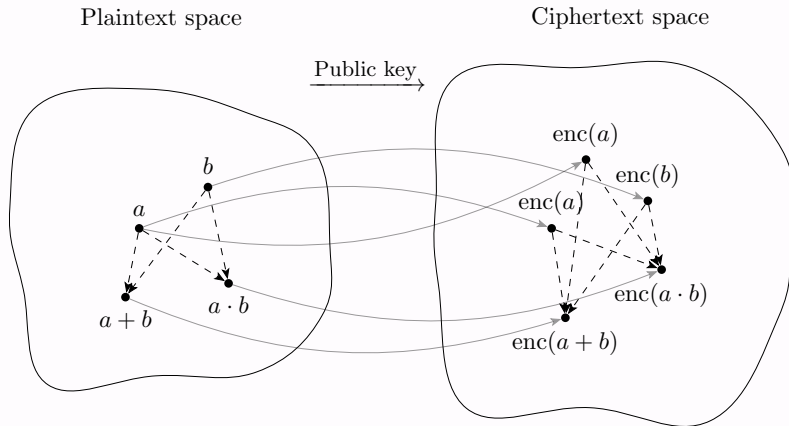
- Fully homomorphic encryption (FHE) allows us to compute functions over encrypted inputs.
- (Almost) any computation can be delegated to a third party without revealing the input x of the function $f(x)$.
- The output is again encrypted and the user can obtain it by decrypting $\text{enc}(f(x))$
- More details avoided¹

¹Thanks Ana :)

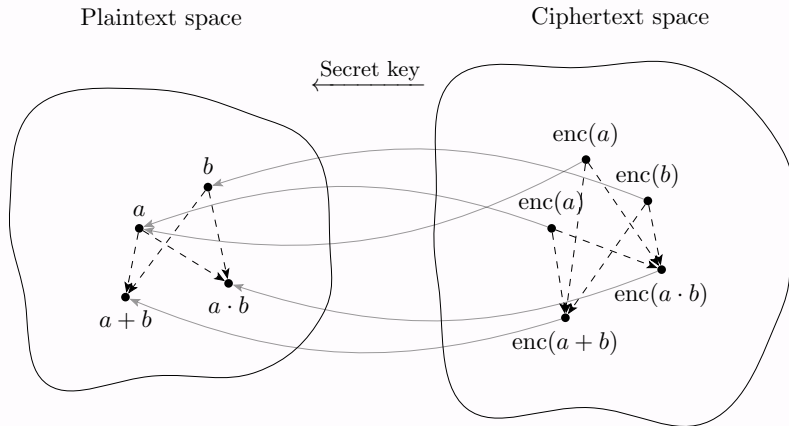
FHE, in a nutshell



FHE, in a nutshell



FHE, in a nutshell



FHE, in a nutshell

- Nowadays, all the main reference FHE schemes are based on the hardness of the ring-Learning with errors (RLWE) problem (they share the same key generation).
- However, the plaintext space and the encoding strategy strongly changes their available operations.
- We can summarize the “classical” strategies as follows:

BGV / (G)BFV
 $\mathbb{Z}_p$

TFHE / FHEW
Small integers

CKKS
 $\mathbb{C}$ (approx.)

Discrete-CKKS
 $\{0,1\} \subset \mathbb{C}$

The problem: large numbers

- Issue: working on large numbers seems hard. Standard definitions of BFV/BGV/TFHE fail in being competitive for a number of reasons.

The problem: large numbers

- Issue: working on large numbers
BFV/BGV/TFHE fail



definitions of
er of reasons.

The problem: large numbers

- Issue: working on large numbers seems hard. Standard definitions of BFV/BGV/TFHE fail in being competitive for a number of reasons.
- Good news: new promising and efficient solutions (e.g., [GV25, Kim25, BK25, CLP25, PZZ25]) are on their way.

The problem: large numbers

- Issue: working on large numbers seems hard. Standard definitions of BFV/BGV/TFHE fail in being competitive for a number of reasons.
- Good news: new promising and efficient solutions (e.g., [GV25, Kim25, BK25, CLP25, PZZ25]) are on their way.

Runtime example

In TFHE-rs a 64-bit multiplication roughly takes ≈ 25 s, 128-bit ≈ 101 s on a single thread (M4 Max). Moreover, it works on 1 slot at the time.

The problem: large numbers

- In this talk we present a *flexible* and *polynomial* version of discrete-CKKS to work over $\mathbb{Z}_{2^k}$ for $k = 2, 4, 8, \dots, 256$.
- Some preliminary results show that latency is competitive w.r.t. the current state of the art (throughput is lower).
- All standard operations ($+$, $\cdot$, $\leq$, $=$, $\ll$) are supported in this framework, although there is some WIP to support advanced operations ($\sqrt{x}$, x^{-1} , $\log(x)$, $\dots$).

Integer arithmetic in CKKS

- All prior works rely on *functional modular reduction* [KN25] during bootstrapping.

Work	Key idea
[Kim25a]	Large int as vector of small chunks
[BK25]	Nested RNS / CRT framework
[Kim25b]	CinS encoding + lazy mod reduction
[CPL25]	Lightweight bootstrapped digit carry

Common limitation

Functional bootstrapping forces an *ad-hoc parameterisation* (1 or 2 levels before bootstrapping). Switching between integer and real computations is not simple.

Defining the problem
○○○○○○○○○○○

Our contribution
●○○○○○○○○○
○○○○○○○○○○○○○○○○○

Some experiments
○○○○○○○

References
○

Our contribution

What we propose

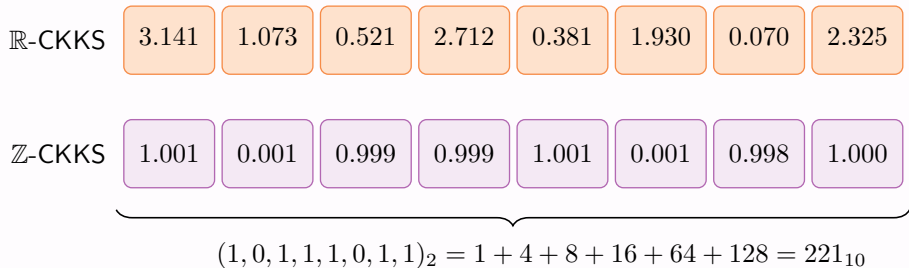
A *fully polynomial* framework for integer arithmetic in CKKS, requiring *no* functional bootstrapping for modular reduction.

- ✓ *Domain-switching*: seamlessly switch between $\mathbb{R}$ -CKKS (reals) and $\mathbb{Z}$ -CKKS (integers, discrete CKKS) with the *same* parameter set.
- ✓ *Standard parameters*: ~ 15 arbitrary multiplicative levels before bootstrapping.
- ✓ *Lowest latency* on all operations (addition, multiplication, comparison, shift) vs. state of the art on large integers (> 64 bits).
- ✗ *Lower throughput* than similar works due to slot overhead for parallel multiplications.

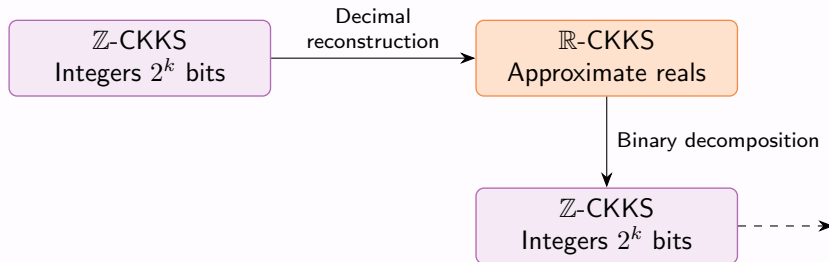
We might call this... Flexible-CKKS :)

$\mathbb{R}$ -CKKS vs $\mathbb{Z}$ -CKKS: structure of the slots

- Visual intuition on why we have less throughput in $\mathbb{Z}$ -CKKS:



Domain-switching



- Possibility: Now we are switching to ℝ-CKKS to speed up integer mults... in the future to speed up arbitrary function evaluations?
- N.B.: By *switching*, we refer to the encoding, and not to *scheme-switching*.

$\mathbb{R}$ -CKKS $\leftrightarrow$ $\mathbb{Z}$ -CKKS: flexible-CKKS!

Easy direction ($\mathbb{Z} \rightarrow \mathbb{R}$):

- Given binary vector $(x_0, \dots, x_{k-1}) \in \{0, 1\}^k$, compute the weighted sum:

$$\sum_{i=0}^{k-1} x_i \cdot 2^i = x$$

via SIMD multiply followed by a rotate-and-sum procedure.

- Cost: $\log(k)$ additions and rotations.

Hard direction ($\mathbb{R} \rightarrow \mathbb{Z}$):

- Given $x \in \mathbb{Z} \cap [0, 2^k)$ in approximate real form, extract *each bit*.
- Prior approach [DMPS23]: evaluate parity function k times (any bits numbers, slow, complexity $O(k)$)
- Our approach: all bits in *one SIMD step* using Fourier series (only 8-bits numbers, fast, complexity $O(1)$).

Binary k -periodic functions

Lemma

The k -th bit of $x \in \mathbb{Z}_{2^k}$ equals $\lfloor x/2^k \rfloor \bmod 2$.

We define an object on top of this lemma.

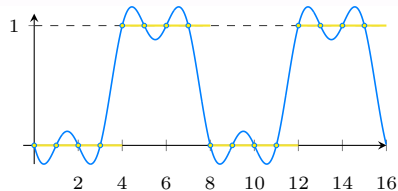
Def. Binary k -periodic function

A function $f : \mathbb{R} \rightarrow \mathbb{R}$ is *binary k -periodic* if for all $x \in \mathbb{Z}$:

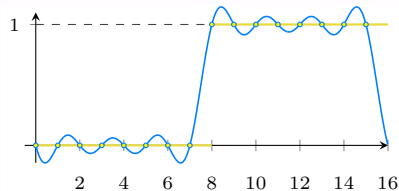
$$f(x) = \lfloor x/2^k \rfloor \bmod 2$$

Basically, a binary k -periodic function can be used to perform a binary decomposition!

Binary k -periodic functions



(a) $k = 2$, allows one to extract the 3rd bit

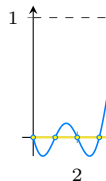


(b) $k = 3$, allows one to extract the 4th bit

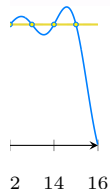
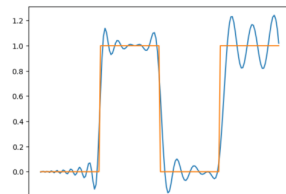
Figure: In yellow the plot of $\lfloor x/2^k \rfloor \bmod 2$, in blue its Fourier expansion derived from a DFT.

E.g. the number 11 in binary is $(1, 1, \mathbf{0}, \mathbf{1})_{10}$

Binary k -periodic functions



Approximating
 $[x/2^k] \bmod 2$



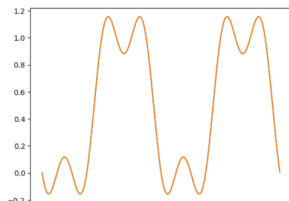
(a) $k = 2, \varepsilon$

Figure: In yell

E.g. the num



Approximating
the interpolating
trigonometric poly

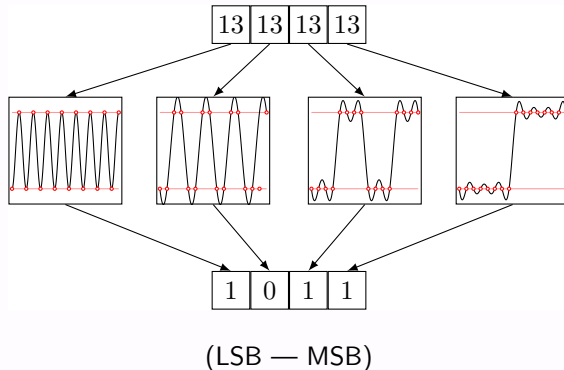


the 4th bit

from a DFT.

Binary k -periodic functions

In general, we can extract the four bits of a number in $[0, 16)$ by evaluating four different polynomials, one for each bit:



Binary k -periodic functions

The pipeline is as follows:

1. Apply DFT to $\lfloor x/2^k \rfloor \bmod 2$ at integers in $[0, 2^k)$ to obtain the *unique* trigonometric polynomial (interpolation).
2. The result is smooth $\Rightarrow$ approximate via *Chebyshev polynomials*.
3. Evaluate using Paterson-Stockmeyer algorithm [PS76, CCS19].

In practice

Every bit of $x + \varepsilon$ for $x \in \mathbb{Z}_{256}$ and small $\varepsilon > 0$ are extracted in one step via SIMD at same cost as evaluating one polynomial. In practice, polynomials of degree 401 return a result $\in \{0, 1\}^8$ with maximum error L_∞ less than 10^{-9} .

Binary k -periodic functions

- Observation: this can be seen as a sort of *polynomial-look-up-table*
- Idea: generalize this to compute the hint of Newton-like algorithms to compute e.g., $\sqrt{x}$, $1/x$ in $\mathbb{R}$ -CKKS, then switch to $\mathbb{Z}$ -CKKS to perform the high-precision iterations.
- Spoiler: some preliminary experiments show that we can efficiently compute e.g. the first 8 bits of the square root $\sqrt{x}$, meaning that with only 4 Newton iterations ($8 \rightarrow 16 \rightarrow 32 \rightarrow 64 \rightarrow 128$) we can compute the square root of a 128-bits number.

Arithmetic Operations in Flexible-CKKS

Additions ($a + b$): the Kogge-Stone Adder (KSA)

We propose to use Kogge-Stone adder (KSA) as it has logarithmic depth, which is ideal for SIMD.

The main algorithm is as follows:

1. Init (1 level):

$$p = a \oplus b, g = a \wedge b$$

2. Parallel prefix ($\log n$ iterations, 1 level per iter.):

$$g := g + (p \cdot \text{Rot}_{-2^d}(g))$$

$$p := p \cdot \text{Rot}_{-2^d}(p)$$

3. Post-process (1 level):

$$c := \text{Rot}_{-1}(g)$$

$$s := ((a - b)^2 - c)^2$$

Lemma 4 (KSA depth)

Adding two n -bit numbers costs

$$2 + \log_2 n$$

CKKS multiplicative levels.
Example: 128-bit addition
 $\Rightarrow$ 9 levels.

Modulo 2 in CKKS

Two approaches to compute $a \oplus b$ in CKKS:

Fast (1 level)

Given $h(x) = x^2$,
Define $a \oplus b = h(a - b)$

- 1 multiplicative level
- Error may *grow*

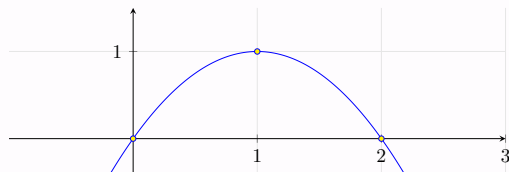
Cleaning (2 levels)

Given $h(x) = x^2(x - 2)^2$,
Define $a \oplus b = h(a + b)$

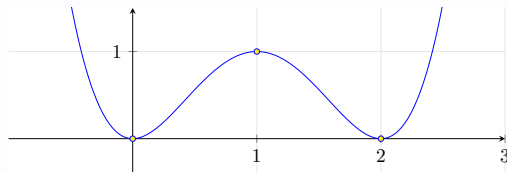
- 2 multiplicative levels
- Error *shrinks quadratically*: $|e'| \leq ke^2$

Modulo 2 in CKKS

Two different approaches to the computation of the modulo 2 operation in CKKS:



(a) The fast XOR introduces a small error



(b) The cleaning XOR reduces the error

Comparisons ($a \leq b$)

Observation

Comparison reduces to *addition* (via subtraction):

$$x \geq y \iff \text{carry-out}(x - \hat{y}) = 1, \quad \hat{y}_i = 1 - y_i$$

The main algorithm is as follows:

1. Invert bits of $\mathbf{b}$ as: $\hat{b}_i = 1 - b_i$
2. Compute $\mathbf{c} = \text{KSA}(\mathbf{a}, \hat{\mathbf{b}})$
3. Return c_n (the carry-out bit)

Therefore comparison has the same complexity as additions.

Logical shifts ($a \ll k$)

Observation

To enable parallel multiplications, every n -bit number is stored followed by $\geq n$ zero slots.

Therefore, a logical shift by k positions is simply:

$$x \ll k \text{ implemented as } \text{Rot}_{-k}(x)$$

Quick comparison:

- [Kim25b]: no efficient way for arbitrary shifts.
- [Kim25a]: 64-bit shift ≈ 100 s.
- [CPL25]: rotations only for multiples of block size B (typically $B = 4$).
- This work: any shift, any size, ≈ 0.10 s.

Multiplications ($a \cdot b$): high-level idea

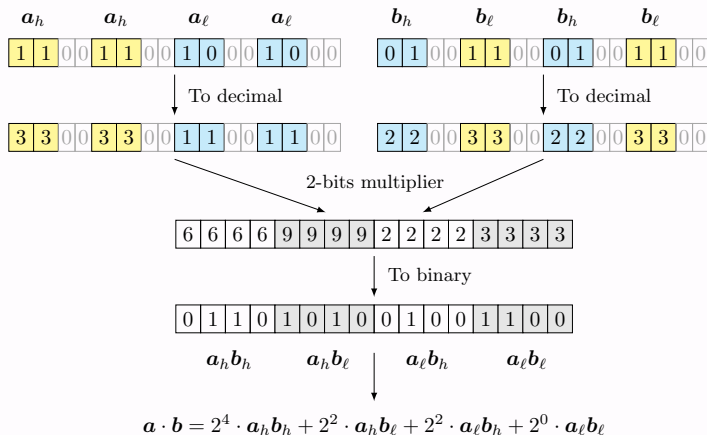


Figure: Reducing $a \cdot b$ to four (shifted) additions. The high bits and low bits are split and repeated for maximum parallelism

Multiplications ($a \cdot b$): high-level idea

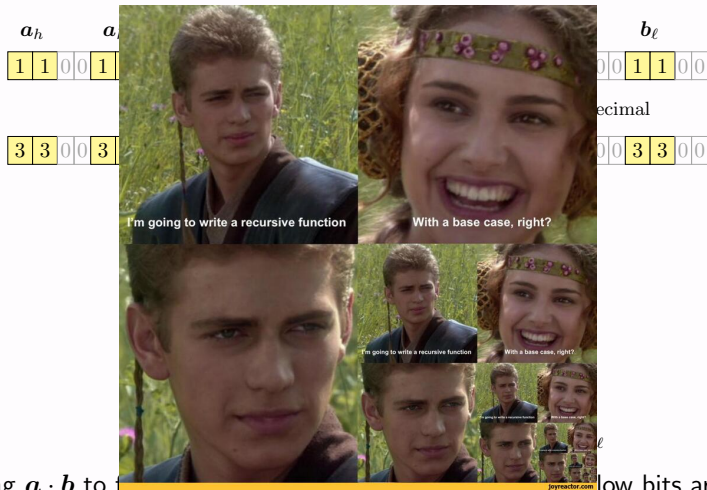


Figure: Reducing $a \cdot b$ to four (limited) additions. The high and low bits are split and repeated for maximum parallelism

Multiplication: divide-et-impera strategy ($n > 4$)

For multiplications, we first split operands into high/low halves:

$$\mathbf{a}, \mathbf{b} \in \{0, 1\}^n \Rightarrow (a_h, a_\ell), (b_h, b_\ell) \in \{0, 1\}^{n/2}$$

Remark that:

$$a \cdot b = 2^n \cdot a_h b_h + 2^{n/2} \cdot a_h b_\ell + 2^{n/2} \cdot a_\ell b_h + a_\ell b_\ell$$

- Four sub-multiplications, all **in parallel** (SIMD)
- Recurse until $n = 4$ (base case)
- Combine partial products via Carry-Save adders (CSA) and KSA

Slot layout for parallel multiplications ($n > 4$)

- Slots double at each level to accommodate the recursive call. This happens as the partial result of a $n \cdot n$ bits product requires n^2 bits:

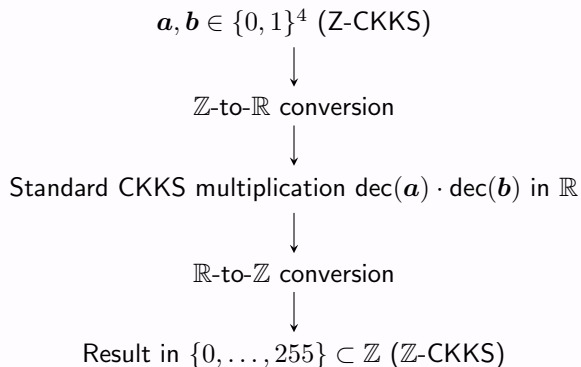
$$\begin{array}{c} n/2 \quad n/2 \\ \boxed{a_\ell} \boxed{0} \cdots \end{array} \quad \text{Slots: } 4n \text{ (case } n = 8 \text{)}$$

$$\begin{array}{c} n/4 \quad n/4 \\ \boxed{(a_\ell)_\ell} \boxed{0} \boxed{(a_\ell)_\ell} \boxed{0} \boxed{(a_\ell)_h} \boxed{0} \boxed{(a_\ell)_h} \boxed{0} \cdots \end{array} \quad \text{Slots: } 8n \text{ (case } n = 16 \text{)}$$

$$\begin{array}{c} n/8 \quad n/8 \\ \boxed{((a_\ell)_\ell)_\ell} \boxed{0} \boxed{((a_\ell)_\ell)_\ell} \boxed{0} \boxed{((a_\ell)_\ell)_h} \boxed{0} \boxed{((a_\ell)_\ell)_h} \boxed{0} \cdots \end{array} \quad \text{Slots: } 16n \text{ (case } n = 32 \text{)}$$

- With s available CKKS slots, one can represent up to $\lfloor 2s/n^2 \rfloor$ distinct n -bit integers compatible with parallel divide-et-impera multiplication.

Multiplication base case ($n = 4$)

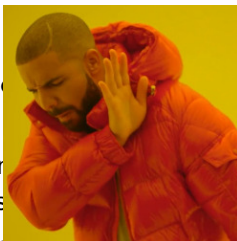


Different bootstrapping types

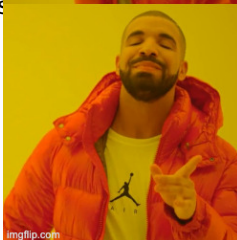
- One last detail: bootstrapping in CKKS allows to restore the modulus back to its original level.
- The main operation is about removing the extra $q_1 \cdot k$ terms that are added during the modulus raise
- The standard bootstrapping [CHKKS18] uses a $\sin$ -like function to simulate a modular reduction

Different bootstrapping types

- One last detail: bootstrapping to the original level.
- The main operation during the modulus reduction
- The standard bootstrapping modular reduction



**standard
ckks
bootstrapping** the modulus back to its
ns that are added
action to simulate a

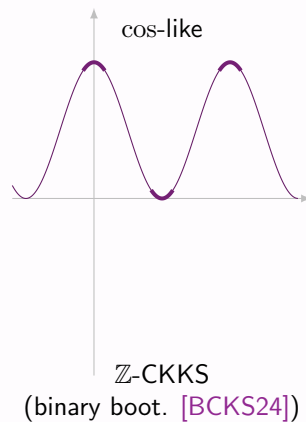
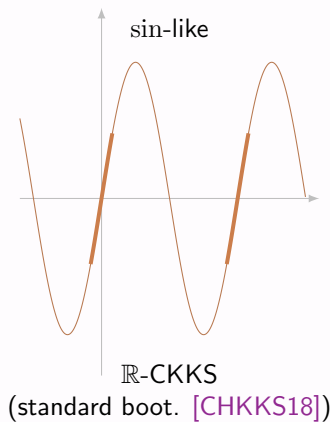


**binary
ckks
bootstrapping**

Different bootstrapping types

- One last detail: bootstrapping in CKKS allows to restore the modulus back to its original level.
- The main operation is about removing the extra $q_1 \cdot k$ terms that are added during the modulus raise
- The standard bootstrapping [CHKKS18] uses a $\sin$ -like function to simulate a modular reduction
- The *binary bootstrapping* [BCKS24] on the other hand uses a $\cos$ -like function with vanishing derivatives to additionally reduce the $\mathbb{Z}$ -CKKS error (e.g., reducing the noise from $0.002 \rightarrow 0.00001$)

Different bootstrapping types



(Image stolen from [Fig. 1, BCKS24])

Defining the problem
○○○○○○○○○○○

Our contribution
○○○○○○○○○○
○○○○○○○○○○○○○○○○

Some experiments
●○○○○○○○

References
○

Some experiments

CKKS Parameters

- Parameters of the scheme:

$\log N$	$\log \Delta$	$\log QP$	d_{num}	$(h, \hat{h})$	Total levels	Arbitrary
16	40	1602	4	(192,32)	30	15

- Key point: the same parameter set supports both $\mathbb{R}$ -CKKS and $\mathbb{Z}$ -CKKS.

Results: Addition $a + b$

This work				[Kim25b]			[CPL25]		
Bits	Time	Slots	BTS	Time	Slots	BTS	Time	Slots	BTS
16	2.12 s	256	1	≈ 12.5 s	8192	2	≈ 13 s	4096	1
32	2.32 s	64	1	≈ 12.5 s	4096	2	≈ 13 s	2048	1
64	2.54 s	16	1	≈ 12.5 s	2048	2	≈ 13 s	1024	1
128	2.61 s	4	1	≈ 12.5 s	1024	2	≈ 13 s	512	1
256	2.89 s	1	1	≈ 12.5 s	512	2	≈ 13 s	256	1

- Our method: 5–6 $\times$ faster and uses only 1 bootstrapping
- Throughput lower (fewer slots), but latency wins decisively
- Times are *expected* for other works, based on their complexity.

Results: Multiplication $a \cdot b$

This work				[Zam22]	[Kim25b]				[CPL25]		
Bits	Time	Slots	BTS	Time	Time	Slots	BTS		Time	Slots	BTS
16	9.2 s	256	2	1.6 s	25.3 s	8192	4		39.9 s	4096	3
32	12.0 s	64	3	6.4 s	24.9 s	4096	4		39.8 s	2048	3
64	16.5 s	16	4	25.4 s	24.9 s	2048	4		40.3 s	1024	3
128	19.3 s	4	5	101 s	30.9 s	1024	5		57.4 s	512	4
256	22.8 s	1	6	403 s	31.9 s	512	5		74.8 s	256	5

- Beats all discrete-CKKS-based alternatives in terms of latency, not on throughput
- At 128 bits: $5\times$ faster than TFHE with 4 slots

Results: Comparisons and Logical Shifts

Comparisons ($a \leq b$):

This work				[Kim25a]		
Bits	Time	Slots	BTS	Time	Slots	BTS
16	2.12 s	256	1	≈ 25 s	16384	8
64	2.54 s	16	1	102 s	16384	32
256	2.89 s	1	1	≈ 395 s	16384	128

Logical shifts:

Method	Cost	Notes
This work	≈ 0.10 s	Any shift, any size
[Kim25a]	≈ 100 s	(64-bit)
[CPL25]	Limited	Only multiples of block $B = 16$

Extra: extension to special primes

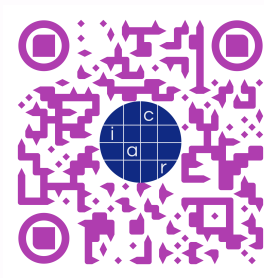
- Our framework natively supports powers of two moduli.
- But many crypto applications use primes like $p = 2^{255} - 19$. Our solution is to follow standard strategies for such moduli:
 1. Work in the surrounding power-of-two space (e.g. 256 bits for $2^{255} - 19$)
 2. After computation, keep the lower 255 bits and multiply the last bit by 19, then add
 3. This requires one additional KSA ($\approx 2\text{--}5$ s overhead)

Comparison with [CPL25]

Using $p = 2^{255} - 19$ in [CPL25] is $> 3\times$ more expensive than their 256-bit baseline. In our framework it costs “only” one extra addition (i.e., ≈ 3 s).

Conclusions

- We proposed a possible construction based on a mixture between standard CKKS and discrete-CKKS.
- You can find the preprint (ia.cr/2026/450), and the source code on GitHub (github.com/lorenzorovida/flexible-integer-arithmetic-ckks):



(a) Preprint IACR ePrint



(b) Source code GitHub

Conclusions

Thank you :-)



References I

- [1] Drucker et al. “Bleach: Cleaning Errors in Discrete Computations over CKKS.” *J. Cryptol.* 37(1), 2023.
- [2] Kim & Noh. “Modular Reduction in CKKS.” *IACR CiC* 2(2), 2025.
- [3] J. Kim. “Efficient Homomorphic Integer Computer from CKKS.” *IACR TCHES* 2025(4).
- [4] J. Kim. “Faster Homomorphic Integer Computer.” ePrint 2025/1440, 2025.
- [5] Boneh & Kim. “Homomorphic Encryption for Large Integers from Nested RNS.” *CRYPTO* 2025.
- [6] Cha, Park & Lee. “Improved Radix-Based Approximate HE for Large Integers.” ePrint 2025/1740, 2025.
- [7] Bae et al. “Bootstrapping Bits with CKKS.”
- [8] Kogge & Stone. “A Parallel Algorithm for Efficient Solution of Recurrence Equations.” *IEEE Trans. Comp.* C-22(8), 1973.
- [9] Zama. “TFHE-rs.” <https://github.com/zama-ai/tfhe-rs>, 2022.